K8S secrets management

Overview of Secrets

To use a Secret, a Pod needs to reference the Secret. A Secret can be used with a Pod in three ways:

- As files in a volume mounted on one or more of its containers.
- As container environment variable.
- By the kubelet when pulling images for the Pod.

The name of a Secret object must be a valid DNS subdomain name. You can specify the `data` and/or the stringData field when creating a configuration file for a Secret. The `data` and the `stringData` fields are optional. The values for all keys in the data field have to be base64-encoded strings. If the conversion to base64 string is not desirable, you can choose to specify the `stringData` field instead, which accepts arbitrary strings as values.

The keys of `data` and `stringData` must consist of alphanumeric characters, -, _ or .. All key-value pairs in the `stringData` field are internally merged into the data field. If a key appears in both the data and the `stringData` field, the value specified in the `stringData` field takes precedence.

Types of Secret

When creating a Secret, you can specify its type using the type field of the `Secret` resource, or certain equivalent `kubectl` command line flags (if available). The Secret type is used to facilitate programmatic handling of the Secret data.

Kubernetes provides several builtin types for some common usage scenarios. These types vary in terms of the validations performed and the constraints Kubernetes imposes on them.

You can define and use your own Secret type by assigning a non-empty string as the type value for a Secret object. An empty string is treated as an Opaque type. Kubernetes doesn't impose any constraints on the type name. However, if you are using one of the builtin types, you must meet all the requirements defined for that type.

Builtin Type	Usage
`Opaque`	arbitrary user-defined data
`kubernetes.io/service-account-token`	service account token
`kubernetes.io/dockercfg`	serialized `~/.dockercfg` file
`kubernetes.io/dockerconfigjson`	serialized `~/.docker/config.json` file
`kubernetes.io/basic-auth`	credentials for basic authentication
`kubernetes.io/ssh-auth`	credentials for SSH authentication
`kubernetes.io/tls`	data for a TLS client or server
`bootstrap.kubernetes.io/token`	bootstrap token data

Opaque secrets

`Opaque` is the default Secret type if omitted from a Secret configuration file. When you create a Secret using `kubectl`, you will use the `generic` subcommand to indicate an `Opaque` Secret type. For example, the following command creates an empty Secret of type `Opaque`.

```
kubectl create secret generic empty-secret
kubectl get secret empty-secret
```

The output looks like:

```
NAME           TYPE     DATA   AGE
empty-secret   Opaque   0      2m6s
```

The `DATA` column shows the number of data items stored in the Secret. In this case, 0 means we have just created an empty Secret.

Service account token Secrets

A `kubernetes.io/service-account-token` type of Secret is used to store a token that identifies a service account. When using this Secret type, you need to ensure that the `kubernetes.io/service-account.name` annotation is set to an existing service account name. A Kubernetes controller fills in some other fields such as the `kubernetes.io/service-account.uid` annotation and the `token` key in the `data` field set to actual token content.

The following example configuration declares a service account token Secret:

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: secret-sa-sample
  annotations:
    kubernetes.io/service-account.name: "sa-name"
type: kubernetes.io/service-account-token
data:
  # You can include additional key value pairs as you do with Opaque Secrets
  extra: YmFyCg==
```

Service account token Secrets

When creating a `Pod`, Kubernetes automatically creates a service account Secret and automatically modifies your Pod to use this Secret. The service account token Secret contains credentials for accessing the API.

The automatic creation and use of API credentials can be disabled or overridden if desired. However, if all you need to do is securely access the API server, this is the recommended workflow.

See the ServiceAccount documentation for more information on how service accounts work. You can also check the automountServiceAccountToken field and the `serviceAccountName` field of the Pod for information on referencing service account from Pods.

Docker config Secrets

You can use one of the following `type` values to create a Secret to store the credentials for accessing a Docker registry for images.

- `kubernetes.io/dockercfg`
- `kubernetes.io/dockerconfigjson`

The `kubernetes.io/dockercfg` type is reserved to store a serialized `~/.dockercfg` which is the legacy format for configuring Docker command line. When using this Secret `type`, you have to ensure the Secret data field contains a `.dockercfg` key whose value is content of a `~/.dockercfg` file encoded in the base64 format.

The `kubernetes.io/dockerconfigjson` type is designed for storing a serialized JSON that follows the same format rules as the `~/.docker/config.json` file which is a new format for `~/.dockercfg`. When using this Secret type, the `data` field of the Secret object must contain a `.dockerconfigjson` key, in which the content for the `~/.docker/config.json` file is provided as a base64 encoded string.

Docker config Secrets

Below is an example for a `kubernetes.io/dockercfg` type of Secret:

```
apiVersion: v1
kind: Secret
metadata:
  name: secret-dockercfg
type: kubernetes.io/dockercfg
data:
  .dockercfg: |
        "<base64 encoded ~/.dockercfg file>"
```

Docker config Secrets

When you create these types of Secrets using a manifest, the API server checks whether the expected key does exists in the `data` field, and it verifies if the value provided can be parsed as a valid JSON. The API server doesn't validate if the JSON actually is a Docker config file.

When you do not have a Docker config file, or you want to use `kubectl` to create a Docker registry Secret, you can do:

```
kubectl create secret docker-registry secret-tiger-docker \
  --docker-username=tiger \
  --docker-password=pass113 \
  --docker-email=tiger@acme.com
```

Docker config Secrets

This command creates a Secret of type `kubernetes.io/dockerconfigjson`. If you dump the `.dockerconfigjson` content from the `data` field, you will get the following JSON content which is a valid Docker configuration created on the fly:

```
{
  "auths": {
    "https://index.docker.io/v1/": {
      "username": "tiger",
      "password": "pass113",
      "email": "tiger@acme.com",
      "auth": "dGlnZXI6cGFzczExMw=="
    }
  }
}
```

Basic authentication Secret

The `kubernetes.io/basic-auth` type is provided for storing credentials needed for basic authentication. When using this Secret type, the `data` field of the Secret must contain the following two keys:

- `username`: the user name for authentication;
- `password`: the password or token for authentication.

Both values for the above two keys are base64 encoded strings. You can, of course, provide the clear text content using the `stringData` for Secret creation.

The following YAML is an example config for a basic authentication Secret:

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: secret-basic-auth
type: kubernetes.io/basic-auth
stringData:
  username: admin
  password: t0p-Secret
```

Basic authentication Secret

The basic authentication Secret type is provided only for user's convenience. You can create an Opaque for credentials used for basic authentication. However, using the builtin Secret type helps unify the formats of your credentials and the API server does verify if the required keys are provided in a Secret configuration.

SSH authentication secrets

The builtin type `kubernetes.io/ssh-auth` is provided for storing data used in SSH authentication. When using this Secret type, you will have to specify a `ssh-privatekey` key-value pair in the `data` (or `stringData`) field as the SSH credential to use.

The following YAML is an example config for a SSH authentication Secret:

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: secret-ssh-auth
type: kubernetes.io/ssh-auth
data:
  # the data is abbreviated in this example
  ssh-privatekey: |
          MIIEpQIBAAKCAQEAulqb/Y ...
```

SSH authentication secrets

The SSH authentication Secret type is provided only for user's convenience. You can create an Opaque for credentials used for SSH authentication. However, using the builtin Secret type helps unify the formats of your credentials and the API server does verify if the required keys are provided in a Secret configuration.

Caution: SSH private keys do not establish trusted communication between an SSH client and host server on their own. A secondary means of establishing trust is needed to mitigate "man in the middle" attacks, such as a known_hosts file added to a ConfigMap.

TLS secrets

Kubernetes provides a builtin Secret type `kubernetes.io/tls` for storing a certificate and its associated key that are typically used for TLS . This data is primarily used with TLS termination of the Ingress resource, but may be used with other resources or directly by a workload. When using this type of Secret, the `tls.key` and the `tls.crt` key must be provided in the `data` (or `stringData`) field of the Secret configuration, although the API server doesn't actually validate the values for each key.

The following YAML contains an example config for a TLS Secret:

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: secret-tls
type: kubernetes.io/tls
data:
  # the data is abbreviated in this example
  tls.crt: |
        MIIC2DCCAcCgAwIBAgIBATANBgkqh ...
  tls.key: |
        MIIEpgIBAAKCAQEA7yn3bRHQ5FHMQ ...
```

TLS secrets

The TLS Secret type is provided for user's convenience. You can create an `Opaque` for credentials used for TLS server and/or client. However, using the builtin Secret type helps ensure the consistency of Secret format in your project; the API server does verify if the required keys are provided in a Secret configuration.

When creating a TLS Secret using `kubectl`, you can use the `tls` subcommand as shown in the following example:

```
kubectl create secret tls my-tls-secret \
  --cert=path/to/cert/file \
  --key=path/to/key/file
```

The public/private key pair must exist before hand. The public key certificate for `--cert` must be .PEM encoded (Base64-encoded DER format), and match the given private key for `--key`. The private key must be in what is commonly called PEM private key format, unencrypted. In both cases, the initial and the last lines from PEM (for example, `--------BEGIN CERTIFICATE-----` and `-------END CERTIFICATE----` for a cetificate) are not included.

Bootstrap token Secrets

A bootstrap token Secret can be created by explicitly specifying the Secret `type` to `bootstrap.kubernetes.io/token`. This type of Secret is designed for tokens used during the node bootstrap process. It stores tokens used to sign well known ConfigMaps.

A bootstrap token Secret is usually created in the `kube-system` namespace and named in the form `bootstrap-token-<token-id>` where `<token-id>` is a 6 character string of the token ID.

As a Kubernetes manifest, a bootstrap token Secret might look like the following:

```
apiVersion: v1
kind: Secret
metadata:
  name: bootstrap-token-5emitj
  namespace: kube-system
type: bootstrap.kubernetes.io/token
data:
  auth-extra-groups: c3lzdGVtOmJvb3RzdHJhcHBlcnM6a3ViZWFkbTpkZWZhdWx0LW5vZGUtdG9rZW4=
  expiration: MjAyMC0wOS0xM1QwNDozOToxMFo=
  token-id: NWVtaXRq
  token-secret: a3E0Z2lodnN6emduMXAwcg==
  usage-bootstrap-authentication: dHJ1ZQ==
  usage-bootstrap-signing: dHJ1ZQ==
```

Bootstrap token Secrets

A bootstrap type Secret has the following keys specified under `data`:

- `token-id`: A random 6 character string as the token identifier. Required.
- `token-secret`: A random 16 character string as the actual token secret. Required.
- `description`: A human-readable string that describes what the token is used for. Optional.
- `expiration`: An absolute UTC time using RFC3339 specifying when the token should be expired. Optional.
- `usage-bootstrap-<usage>`: A boolean flag indicating additional usage for the bootstrap token.
- `auth-extra-groups`: A comma-separated list of group names that will be authenticated as in addition to the `system:bootstrappers` group.

Bootstrap token Secrets

The above YAML may look confusing because the values are all in base64 encoded strings. In fact, you can create an identical Secret using the following YAML:

```yaml
apiVersion: v1
kind: Secret
metadata:
  # Note how the Secret is named
  name: bootstrap-token-5emitj
  # A bootstrap token Secret usually resides in the kube-system namespace
  namespace: kube-system
type: bootstrap.kubernetes.io/token
stringData:
  auth-extra-groups: "system:bootstrappers:kubeadm:default-node-token"
  expiration: "2020-09-13T04:39:10Z"
  # This token ID is used in the name
  token-id: "5emitj"
  token-secret: "kq4gihvszzgn1p0r"
  # This token can be used for authentication
  usage-bootstrap-authentication: "true"
  # and it can be used for signing
  usage-bootstrap-signing: "true"
```

Creating a Secret

There are several options to create a Secret:

- create Secret using kubectl command
- create Secret from config file
- create Secret using kustomize

Editing a Secret

An existing Secret may be edited with the following command:

```
kubectl edit secrets mysecret
```

This will open the default configured editor and allow for updating the base64 encoded Secret values in the `data` field:

```
# Please edit the object below. Lines beginning with a '#' will be ignored,
# and an empty file will abort the edit. If an error occurs while saving this file will be
# reopened with the relevant failures.
#
apiVersion: v1
data:
  username: YWRtaW4=
  password: MWYyZDFlMmU2N2Rm
kind: Secret
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: { ... }
  creationTimestamp: 2016-01-22T18:41:56Z
  name: mysecret
  namespace: default
  resourceVersion: "164619"
  uid: cfee02d6-c137-11e5-8d73-42010af00002
type: Opaque
```

Using Secrets

Secrets can be mounted as data volumes or exposed as environment variables to be used by a container in a Pod. Secrets can also be used by other parts of the system, without being directly exposed to the Pod. For example, Secrets can hold credentials that other parts of the system should use to interact with external systems on your behalf.

Using Secrets as files from a Pod

To consume a Secret in a volume in a Pod:

1. Create a secret or use an existing one. Multiple Pods can reference the same secret.
2. Modify your Pod definition to add a volume under `.spec.volumes[]`. Name the volume anything, and have a `.spec.volumes[].secret.secretName` field equal to the name of the Secret object.

Using Secrets

3. Add a `.spec.containers[].volumeMounts[]` to each container that needs the secret. Specify `.spec.containers[].volumeMounts[].readOnly = true` and `.spec.containers[].volumeMounts[].mountPath` to an unused directory name where you would like the secrets to appear.
4. Modify your image or command line so that the program looks for files in that directory. Each key in the secret `data` map becomes the filename under `mountPath`.

This is an example of a Pod that mounts a Secret in a volume:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  containers:
  - name: mypod
    image: redis
    volumeMounts:
    - name: foo
      mountPath: "/etc/foo"
      readOnly: true
  volumes:
  - name: foo
    secret:
      secretName: mysecret
```

Using Secrets

Each Secret you want to use needs to be referred to in `.spec.volumes`.

If there are multiple containers in the Pod, then each container needs its own `volumeMounts` block, but only one `.spec.volumes` is needed per Secret.

You can package many files into one secret, or use many secrets, whichever is convenient.

Projection of Secret keys to specific paths

You can also control the paths within the volume where Secret keys are projected. You can use the `.spec.volumes[].secret.items` field to change the target path of each key:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  containers:
  - name: mypod
    image: redis
    volumeMounts:
    - name: foo
      mountPath: "/etc/foo"
      readOnly: true
  volumes:
  - name: foo
    secret:
      secretName: mysecret
      items:
      - key: username
        path: my-group/my-username
```

Projection of Secret keys to specific paths

What will happen:

- `username` secret is stored under `/etc/foo/my-group/my-username` file instead of `/etc/foo/username`.
- `password` secret is not projected.

If `.spec.volumes[].secret.items` is used, only keys specified in `items` are projected. To consume all keys from the secret, all of them must be listed in the `items` field. All listed keys must exist in the corresponding secret. Otherwise, the volume is not created.

Projection of Secret keys to specific paths

Secret files permissions

You can set the file access permission bits for a single Secret key. If you don't specify any permissions, `0644` is used by default. You can also set a default mode for the entire Secret volume and override per key if needed.

For example, you can specify a default mode like this:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  containers:
  - name: mypod
    image: redis
    volumeMounts:
    - name: foo
      mountPath: "/etc/foo"
  volumes:
  - name: foo
    secret:
      secretName: mysecret
      defaultMode: 0400
```

Projection of Secret keys to specific paths

Then, the secret will be mounted on `/etc/foo` and all the files created by the secret volume mount will have permission `0400`.

Note that the JSON spec doesn't support octal notation, so use the value 256 for 0400 permissions. If you use YAML instead of JSON for the Pod, you can use octal notation to specify permissions in a more natural way.

Note if you `kubectl exec` into the Pod, you need to follow the symlink to find the expected file mode. For example,

Check the secrets file mode on the pod.

```
kubectl exec mypod -it sh

cd /etc/foo
ls -l
```

Projection of Secret keys to specific paths

The output is similar to this:

```
total 0
lrwxrwxrwx 1 root root 15 May 18 00:18 password -> ..data/password
lrwxrwxrwx 1 root root 15 May 18 00:18 username -> ..data/username
```

Follow the symlink to find the correct file mode.

```
cd /etc/foo/..data
ls -l
```

The output is similar to this:

```
total 8
-r-------- 1 root root 12 May 18 00:18 password
-r-------- 1 root root  5 May 18 00:18 username
```

Projection of Secret keys to specific paths

You can also use mapping, as in the previous example, and specify different permissions for different files like this:

In this case, the file resulting in `/etc/foo/my-group/my-username` will have permission value of `0777`. If you use JSON, owing to JSON limitations, you must specify the mode in decimal notation, `511`. Note that this permission value might be displayed in decimal notation if you read it later.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  containers:
  - name: mypod
    image: redis
    volumeMounts:
    - name: foo
      mountPath: "/etc/foo"
  volumes:
  - name: foo
    secret:
      secretName: mysecret
      items:
      - key: username
        path: my-group/my-username
        mode: 0777
```

Consuming Secret values from volumes

Inside the container that mounts a secret volume, the secret keys appear as files and the secret values are base64 decoded and stored inside these files. This is the result of commands executed inside the container from the example above:

```
ls /etc/foo/
```

The output is similar to:

```
username
password
```

```
cat /etc/foo/username
```

The output is similar to:

```
admin
```

```
cat /etc/foo/password
```

The output is similar to:

```
1f2d1e2e67df
```

Mounted Secrets are updated automatically

When a secret currently consumed in a volume is updated, projected keys are eventually updated as well. The kubelet checks whether the mounted secret is fresh on every periodic sync. However, the kubelet uses its local cache for getting the current value of the Secret. The type of the cache is configurable using the `ConfigMapAndSecretChangeDetectionStrategy` field in the KubeletConfiguration struct. A Secret can be either propagated by watch (default), ttl-based, or by redirecting all requests directly to the API server. As a result, the total delay from the moment when the Secret is updated to the moment when new keys are projected to the Pod can be as long as the kubelet sync period + cache propagation delay, where the cache propagation delay depends on the chosen cache type (it equals to watch propagation delay, ttl of cache, or zero correspondingly).

Using Secrets as environment variables

To use a secret in an environment variable in a Pod:

1. Create a secret or use an existing one. Multiple Pods can reference the same secret.
2. Modify your Pod definition in each container that you wish to consume the value of a secret key to add an environment variable for each secret key you wish to consume. The environment variable that consumes the secret key should populate the secret's name and key in env[].valueFrom.secretKeyRef.
3. Modify your image and/or command line so that the program looks for values in the specified environment variables.

This is an example of a Pod that uses secrets from environment variables:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: secret-env-pod
spec:
  containers:
  - name: mycontainer
    image: redis
    env:
      - name: SECRET_USERNAME
        valueFrom:
          secretKeyRef:
            name: mysecret
            key: username
      - name: SECRET_PASSWORD
        valueFrom:
          secretKeyRef:
            name: mysecret
            key: password
  restartPolicy: Never
```

Consuming Secret Values from environment variables

Inside a container that consumes a secret in an environment variables, the secret keys appear as normal environment variables containing the base64 decoded values of the secret data. This is the result of commands executed inside the container from the example above:

```
echo $SECRET_USERNAME
```

The output is similar to:

```
admin
```

```
echo $SECRET_PASSWORD
```

The output is similar to:

```
1f2d1e2e67df
```

Environment variables are not updated after a secret update

If a container already consumes a Secret in an environment variable, a Secret update will not be seen by the container unless it is restarted. There are third party solutions for triggering restarts when secrets change.

Immutable Secrets

FEATURE STATE: **Kubernetes v1.19 [beta]**

The Kubernetes beta feature Immutable Secrets and ConfigMaps provides an option to set individual Secrets and ConfigMaps as immutable. For clusters that extensively use Secrets (at least tens of thousands of unique Secret to Pod mounts), preventing changes to their data has the following advantages:

- protects you from accidental (or unwanted) updates that could cause applications outages
- improves performance of your cluster by significantly reducing load on kube-apiserver, by closing watches for secrets marked as immutable.

Immutable Secrets

This feature is controlled by the ImmutableEphemeralVolumes feature gate, which is enabled by default since v1.19. You can create an immutable Secret by setting the immutable field to true. For example,

```yaml
apiVersion: v1
kind: Secret
metadata:
  ...
data:
  ...
immutable: true
```

Immutable Secrets

Using imagePullSecrets

The `imagePullSecrets` field is a list of references to secrets in the same namespace. You can use an `imagePullSecrets` to pass a secret that contains a Docker (or other) image registry password to the kubelet. The `kubelet` uses this information to pull a private image on behalf of your Pod.

Arranging for imagePullSecrets to be automatically attached

You can manually create `imagePullSecrets`, and reference it from a ServiceAccount. Any Pods created with that ServiceAccount or created with that ServiceAccount by default, will get their `imagePullSecrets` field set to that of the service account.

Details

Restrictions

Secret volume sources are validated to ensure that the specified object reference actually points to an object of type Secret. Therefore, a secret needs to be created before any Pods that depend on it.

Secret resources reside in a namespace. Secrets can only be referenced by Pods in that same namespace.

Individual secrets are limited to 1MiB in size. This is to discourage creation of very large secrets which would exhaust the API server and kubelet memory. However, creation of many smaller secrets could also exhaust memory. More comprehensive limits on memory usage due to secrets is a planned feature.

Details

The kubelet only supports the use of secrets for Pods where the secrets are obtained from the API server. This includes any Pods created using `kubectl`, or indirectly via a replication controller. It does not include Pods created as a result of the kubelet `--manifest-url` flag, its `--config` flag, or its REST API (these are not common ways to create Pods.)

Secrets must be created before they are consumed in Pods as environment variables unless they are marked as optional. References to secrets that do not exist will prevent the Pod from starting.

References (`secretKeyRef` field) to keys that do not exist in a named Secret will prevent the Pod from starting.

Details

Secrets used to populate environment variables by the `envFrom` field that have keys that are considered invalid environment variable names will have those keys skipped. The Pod will be allowed to start. There will be an event whose reason is `InvalidVariableNames` and the message will contain the list of invalid keys that were skipped. The example shows a pod which refers to the default/mysecret that contains 2 invalid keys: `1badkey` and `2alsobad`.

```
kubectl get events
```

The output is similar to:

```
LASTSEEN   FIRSTSEEN   COUNT     NAME            KIND      SUBOBJECT
0s         0s          1         dapi-test-pod   Pod
```

```
TYPE      REASON
Warning   InvalidEnvironmentVariableNames   kubelet, 127.0.0.1      Keys [1badkey, 2alsobad]

from the EnvFrom secret default/mysecret were skipped since they are considered
invalid environment variable names.
```

Details

Secret and Pod lifetime interaction

When a Pod is created by calling the Kubernetes API, there is no check if a referenced secret exists. Once a Pod is scheduled, the kubelet will try to fetch the secret value. If the secret cannot be fetched because it does not exist or because of a temporary lack of connection to the API server, the kubelet will periodically retry. It will report an event about the Pod explaining the reason it is not started yet. Once the secret is fetched, the kubelet will create and mount a volume containing it. None of the Pod's containers will start until all the Pod's volumes are mounted.

Use cases

Use-Case: As container environment variables

Create a secret

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: mysecret
type: Opaque
data:
  USER_NAME: YWRtaW4=
  PASSWORD: MWYyZDFlMmU2N2Rm
```

Create the Secret:

```
kubectl apply -f mysecret.yaml
```

Use cases

Use `envFrom` to define all of the Secret's data as container environment variables. The key from the Secret becomes the environment variable name in the Pod.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: secret-test-pod
spec:
  containers:
    - name: test-container
      image: k8s.gcr.io/busybox
      command: [ "/bin/sh", "-c", "env" ]
      envFrom:
      - secretRef:
          name: mysecret
  restartPolicy: Never
```

Use cases

Use-Case: Pod with ssh keys

Create a secret containing some ssh keys:

```
kubectl create secret generic ssh-key-secret -from-file=ssh-privatekey=/path/to/.ssh/id_rsa
--from-file=ssh-publickey=/path/to/.ssh/id_rsa.pub
```

The output is similar to: `secret "ssh-key-secret" created`

You can also create a `kustomization.yaml` with a `secretGenerator` field containing ssh keys.

Use cases

Now you can create a Pod which references the secret with the ssh key and consumes it in a volume:

When the container's command runs, the pieces of the key will be available in:

```
/etc/secret-volume/ssh-publickey
/etc/secret-volume/ssh-privatekey
```

The container is then free to use the secret data to establish an ssh connection.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: secret-test-pod
  labels:
    name: secret-test
spec:
  volumes:
  - name: secret-volume
    secret:
      secretName: ssh-key-secret
  containers:
  - name: ssh-test-container
    image: mySshImage
    volumeMounts:
    - name: secret-volume
      readOnly: true
      mountPath: "/etc/secret-volume"
```

Use cases

Use-Case: Pods with prod / test credentials

This example illustrates a Pod which consumes a secret containing production credentials and another Pod which consumes a secret with test environment credentials.

You can create a `kustomization.yaml` with a `secretGenerator` field or run `kubectl` create secret.

```
kubectl create secret generic prod-db-secret --from-literal=username=produser --from-literal=password=Y4nys7f11
```

The output is similar to: `secret "prod-db-secret" created`

Use cases

Use-Case: Pods with prod / test credentials

You can also create a secret for test environment credentials.

```
kubectl create secret generic test-db-secret --from-literal=username=testuser --from-literal=password=iluvtests
```

The output is similar to: `secret "test-db-secret" created`

Use cases

Now make the Pods:

```
cat <<EOF > pod.yaml
apiVersion: v1
kind: List
items:
- kind: Pod
  apiVersion: v1
  metadata:
    name: prod-db-client-pod
    labels:
      name: prod-db-client
  spec:
    volumes:
    - name: secret-volume
      secret:
        secretName: prod-db-secret
    containers:
    - name: db-client-container
      image: myClientImage
      volumeMounts:
      - name: secret-volume
        readOnly: true
        mountPath: "/etc/secret-volume"
- kind: Pod
  apiVersion: v1
  metadata:
    name: test-db-client-pod
    labels:
      name: test-db-client
  spec:
    volumes:
    - name: secret-volume
      secret:
        secretName: test-db-secret
    containers:
    - name: db-client-container
      image: myClientImage
      volumeMounts:
      - name: secret-volume
        readOnly: true
        mountPath: "/etc/secret-volume"
EOF
```

Use cases

Add the pods to the same kustomization.yaml:

```
cat <<EOF >> kustomization.yaml
resources:
- pod.yaml
EOF
```

Apply all those objects on the API server by running:

```
kubectl apply -k .
```

Both containers will have the following files present on their filesystems with the values for each container's environment:

```
/etc/secret-volume/username
/etc/secret-volume/password
```

Note how the specs for the two Pods differ only in one field; this facilitates creating Pods with different capabilities from a common Pod template.

Use cases

You could further simplify the base Pod specification by using two service accounts:

1. `prod-user` with the `prod-db-secret`
2. `test-user` with the `test-db-secret`

The Pod specification is shortened to:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: prod-db-client-pod
  labels:
    name: prod-db-client
spec:
  serviceAccount: prod-db-client
  containers:
  - name: db-client-container
    image: myClientImage
```

Use cases

Use-case: dotfiles in a secret volume

You can make your data "hidden" by defining a key that begins with a dot. This key represents a dotfile or "hidden" file. For example, when the following secret is mounted into a volume, `secret-volume`:

The volume will contain a single file, called `.secret-file`, and the `dotfile-test-container` will have this file present at the path `/etc/secret-volume/.secret-file`.

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: dotfile-secret
data:
  .secret-file: dmFsdWUtMg0KDQo=
---
apiVersion: v1
kind: Pod
metadata:
  name: secret-dotfiles-pod
spec:
  volumes:
  - name: secret-volume
    secret:
      secretName: dotfile-secret
  containers:
  - name: dotfile-test-container
    image: k8s.gcr.io/busybox
    command:
    - ls
    - "-l"
    - "/etc/secret-volume"
    volumeMounts:
    - name: secret-volume
      readOnly: true
      mountPath: "/etc/secret-volume"
```

Use cases

Use-case: Secret visible to one container in a Pod

Consider a program that needs to handle HTTP requests, do some complex business logic, and then sign some messages with an HMAC. Because it has complex application logic, there might be an unnoticed remote file reading exploit in the server, which could expose the private key to an attacker.

This could be divided into two processes in two containers: a frontend container which handles user interaction and business logic, but which cannot see the private key; and a signer container that can see the private key, and responds to simple signing requests from the frontend (for example, over localhost networking).

With this partitioned approach, an attacker now has to trick the application server into doing something rather arbitrary, which may be harder than getting it to read a file.

Best practices

Clients that use the Secret API

When deploying applications that interact with the Secret API, you should limit access using authorization policies such as RBAC.

Secrets often hold values that span a spectrum of importance, many of which can cause escalations within Kubernetes (e.g. service account tokens) and to external systems. Even if an individual app can reason about the power of the secrets it expects to interact with, other apps within the same namespace can render those assumptions invalid.

For these reasons watch and list requests for secrets within a namespace are extremely powerful capabilities and should be avoided, since listing secrets allows the clients to inspect the values of all secrets that are in that namespace. The ability to watch and list all secrets in a cluster should be reserved for only the most privileged, system-level components.

Best practices

Applications that need to access the Secret API should perform get requests on the secrets they need. This lets administrators restrict access to all secrets while white-listing access to individual instances that the app needs.

For improved performance over a looping get, clients can design resources that reference a secret then watch the resource, re-requesting the secret when the reference changes. Additionally, a "bulk watch" API to let clients watch individual resources has also been proposed, and will likely be available in future releases of Kubernetes.

Security properties

Protections

Because secrets can be created independently of the Pods that use them, there is less risk of the secret being exposed during the workflow of creating, viewing, and editing Pods. The system can also take additional precautions with Secrets, such as avoiding writing them to disk where possible.

A secret is only sent to a node if a Pod on that node requires it. The kubelet stores the secret into a tmpfs so that the secret is not written to disk storage. Once the Pod that depends on the secret is deleted, the kubelet will delete its local copy of the secret data as well.

There may be secrets for several Pods on the same node. However, only the secrets that a Pod requests are potentially visible within its containers. Therefore, one Pod does not have access to the secrets of another Pod.

Security properties

Protections

There may be several containers in a Pod. However, each container in a Pod has to request the secret volume in its volumeMounts for it to be visible within the container. This can be used to construct useful security partitions at the Pod level.

On most Kubernetes distributions, communication between users and the API server, and from the API server to the kubelets, is protected by SSL/TLS. Secrets are protected when transmitted over these channels.

FEATURE STATE: **Kubernetes v1.13 [beta]**

You can enable encryption at rest for secret data, so that the secrets are not stored in the clear into etcd.

Security properties

Risks

- In the API server, secret data is stored in etcd; therefore:
  - Administrators should enable encryption at rest for cluster data (requires v1.13 or later).
  - Administrators should limit access to etcd to admin users.
  - Administrators may want to wipe/shred disks used by etcd when no longer in use.
  - If running etcd in a cluster, administrators should make sure to use SSL/TLS for etcd peer-to-peer communication.
- If you configure the secret through a manifest (JSON or YAML) file which has the secret data encoded as base64, sharing this file or checking it in to a source repository means the secret is compromised. Base64 encoding is not an encryption method and is considered the same as plain text.
- Applications still need to protect the value of secret after reading it from the volume, such as not accidentally logging it or transmitting it to an untrusted party.
- A user who can create a Pod that uses a secret can also see the value of that secret. Even if the API server policy does not allow that user to read the Secret, the user could run a Pod which exposes the secret.
- Currently, anyone with root permission on any node can read any secret from the API server, by impersonating the kubelet. It is a planned feature to only send secrets to nodes that actually require them, to restrict the impact of a root exploit on a single node.

Q&A